

MTGNP v1.0 — Magic: The Gathering Network Protocol

CSNETWK Machine Problem — implementation of RFC 0001 (MTGNP v1.0).

Contents

File	Purpose
<code>server.py</code>	Authoritative Game Server (state machines, rules engine)
<code>client.py</code>	Interactive Player Client (thin renderer, RFC 4.3)
<code>gui_client.py</code>	GUI Player Client (web-based interface wrapper over <code>client.py</code>)
<code>spectator.py</code>	Spectator Client (read-only observer, bonus feature)
<code>common.py</code>	Shared framing (4-byte BE length prefix + JSON), verbose logger, catalog loader
<code>cards.json</code>	Shared card catalog, pre-loaded out-of-band by both sides (RFC Sec. 1)
<code>gui/index.html</code>	Web GUI frontend (cyberpunk-themed game interface)
<code>launch_gui_game.py</code>	Launcher script (starts server + 2 GUI clients + opens browsers)
<code>decks/burn.txt</code> , <code>decks/control.txt</code>	Sample deck lists (one card instance ID per line)
<code>test_game.py</code>	End-to-end protocol test (lobby, mulligan, errors, concede, restart)
<code>test_combat.py</code>	Deep-mechanics test (haste, blocking trade, counterspell, triggers)
<code>test_edge.py</code>	Edge cases: damage order, first strike, trigger ordering, deck-out, timeout

Project structure

```

mtgnp/
├── decks/
│   ├── burn.txt          # sample red aggro deck list
│   └── control.txt       # sample blue/black control deck list
├── gui/
│   └── index.html        # web GUI frontend (cyberpunk-themed)
├── README.md             # this document (Markdown source)
├── README.pdf            # this document (submission PDF)
├── cards.json            # shared card catalog (loaded by server and client)
├── client.py             # Player Client - entry point: python3 client.py
├── common.py            # shared framing / logging / catalog helpers
├── gui_client.py         # GUI Client - entry point: python3 gui_client.py
├── launch_gui_game.py    # Launcher - starts server + 2 GUI clients
├── server.py            # Game Server - entry point: python3 server.py
├── spectator.py         # Spectator - read-only observer client
├── test_combat.py        # deep-mechanics test suite (3 assertions)
├── test_edge.py          # edge-case test suite (14 assertions)
└── test_game.py          # end-to-end protocol test suite (10 assertions)

```

Requirements: **Python 3.10+**, standard library only. No third-party packages.

Running — step by step

Prerequisites: Python 3.10 or newer. No installation of packages is needed (standard library only).

Windows note: the command is `python` (or `py`), *not* `python3` — substitute it in every command below, e.g. `python server.py --verbose`. If Windows says "Python was not found", install Python from python.org/downloads and tick "**Add python.exe to PATH**" in the installer, then reopen the terminal. Check with `python --version` (Linux/macOS: `python3 --version`).

1. **Unzip the project** and open a terminal in the project folder (the one containing `server.py`).
2. **Start the server** (Terminal 1). It listens on port 4444 (RFC 5.1):

```
python3 server.py --verbose
```

You should see `[server]` listening on port 4444. Use `--port <n>` if 4444 is taken (then add `--port <n>` to the clients too).

3. **Start the first client** (Terminal 2):

```
python3 client.py --id player_1 --deck decks/burn.txt --verbose
```

If the server is on another machine, add `--host <server-ip>`.

4. **Start the second client** (Terminal 3):

```
python3 client.py --id player_2 --deck decks/control.txt --verbose
```

A third connection attempt will be refused by the server (RFC 5.1).

5. **Join the game:** type `ready` in each client and press Enter. When both are ready the server deals hands and the mulligan begins.

6. **Mulligan:** type `keep` to keep your hand, or `mull` to redraw. After N mulligans, keep with `keep <card_1> ... <card_N>` to put N cards on the bottom of your library (London mulligan).

7. **Play:** when you see `>>> You have priority`, you may act:

- `play mountain_003` — play a land (your main phase only, 1/turn)
- `cast lightning_bolt_001 player_2` — cast a spell (mana is paid automatically from your untapped lands)
- `pass` — pass priority
- `attack goblin_guide_001` — declare attackers (in DECLARE_ATTACKERS)
- `block wall_of_stone_001:goblin_guide_001` — declare blockers
- `order <attacker> <blocker1> <blocker2>` — damage order (multi-blocks)
- `discard <card>` — discard to 7 at cleanup; `yes / no` — answer an optional trigger; `hand / state` — re-print; `concede` — give up; `help` — full list

8. **Game over:** the winner and reason are announced; both clients stay connected and can type `ready` to start a new game (RFC 6.6).

Verbose mode (rubric prerequisite)

`--verbose / -v` on **either program** prints **every PDU sent and received**, labelled with direction, peer, timestamp, and the full JSON:

```
[14:03:22] SEND  player_1 | {"type": "PRIORITY_GRANT", "player_id": "player_1", ...}
[14:03:23] RECV  player_1 | {"type": "CAST_SPELL", "seq_num": 12, ...}
```

Without the flag both programs run quietly. This satisfies the "Verbose Mode Requirement" toggle in the project specification.

Debug flags (server)

- `--time-limit <ms>` — priority deadline (default 60000; RFC 4.2)
- `--seed <n>` — seed the RNG for reproducible shuffles/coin flips (demos)
- `--first {0,1}` — force which seat goes first (demos/tests)
- `--port <n>` — listen port

Tests

```
python3 test_game.py      # 10 protocol assertions
python3 test_combat.py    # 3 combat/effect assertions
python3 test_edge.py      # 18 edge-case assertions (multi-block damage order,
                           # first strike, WRONG_PHASE / INSUFFICIENT_MANA /
                           # ILLEGAL_ACTION + same-seq re-grant, Gray Merchant
                           # drain, TRIGGER_ORDER with simultaneous death
                           # triggers, DECK_EMPTY, priority-timeout DISCONNECT)
```

`test_edge.py` derives a deterministic RNG seed by replicating the server's seeded shuffle locally, so its scripted 10-turn game is fully reproducible.

Design summary

- **Transport (RFC 5):** every PDU is a 4-byte big-endian length prefix + UTF-8 JSON, max 65,535 bytes.
`common.recv_pdu` reads exactly the framed bytes before parsing; malformed payloads yield `ERROR INVALID_JSON`.
- **Threads:** the server runs one reader thread per client feeding a single event queue; the main thread runs the RFC Section 6 lifecycle (`LOBBY` → `GAME_SETUP` → `MULLIGAN` → `IN_GAME` → `GAME_OVER` → `LOBBY`, same TCP connections). `PING` is answered with `PONG` directly in the reader thread (independent heartbeat counter, RFC 5.4). The client runs reader, heartbeat (`PING/30 s`, `10 s PONG` deadline) and `stdin` threads.
- **Authority (RFC 4.2/4.3):** all rules live in the server. The client never simulates outcomes; each personalized `GAME_STATE_UPDATE` (own hand only, opponent hand as a count) overwrites its view.
- **seq_num discipline (RFC 5.4):** the server keeps one monotonically increasing counter; a broadcast consumes one number. The client echoes the seq of the latest `PRIORITY_GRANT` for priority actions, the relevant request PDU for `MULLIGAN_CHOICE/DISCARD` (the `GAME_STATE_UPDATE`) and combat declarations (the `PHASE_TRANSITION`), and any last server seq for `CONCEDE`. Mismatches get `ERROR STALE_ACTION` and, when the player still holds priority, a re-granted token. After an *illegal* (non-stale) action the server re-issues `PRIORITY_GRANT` with the **same** seq_num (RFC Sec. 11 item 3).
- **Rules implemented:** London mulligan; full phase sequence with first-turn draw skip; one land per turn at sorcery speed; implicit mana (declared `mana_payment` validated against untapped lands, which are then tapped); LIFO stack with priority windows and caster-retains-priority; fizzle on illegal targets at resolution; state-based actions after every event (lethal damage, 0 toughness, $\text{life} \leq 0$, AP loses simultaneous-death ties); summoning sickness; combat with attack tapping, single-attacker blocks, multi-block damage ordering (`ASSIGN_DAMAGE_ORDER`), first-strike step when relevant, no trample; cleanup discard-to-7 loop; win by `LIFE_ZERO`, `DECK_EMPTY`, `CONCEDE`, `DISCONNECT` (incl. priority timeout); extra connections refused; lobby deck resubmission; duplicate ID rejection.
- **Card effects (≥ 5 , RFC Appendix set):** Lightning Bolt / Shock (damage), Counterspell (counters a stack item), Giant Growth (+3/+3 until EOT), Healing Salve (lifegain), Divination (draw 2), Gray Merchant of Asphodel (ETB drain-2 optional **triggered ability** exercised through `TRIGGER_CHOICE` / `TRIGGER_CHOICE_RESPONSE`), Festering Imp (death trigger; two dying simultaneously exercises `TRIGGER_ORDER` /

TRIGGER_ORDER_RESPONSE), Goblin Guide (haste), Youthful Knight (first strike), plus vanilla creatures and five basic lands.

Known deviations / interpretations

- A countered spell is announced with `STACK_RESOLVE result: "FIZZLE"` (the RFC only defines `RESOLVED | FIZZLE`, with no dedicated "countered" result).
- Gray Merchant's drain is modelled as an *optional* ("you may") trigger so the `TRIGGER_CHOICE` flow of RFC 8.6.3 is exercised; declining discards it.
- `ACTIVATE_ABILITY` is answered with `ILLEGAL_ACTION`: the fixed card set defines no activated non-mana abilities, and mana is implicit (RFC 7.5). As a result the interactive client has no command that sends it (the server-side rejection path is still fully implemented).
- Summoning sickness is cleared for the active player's creatures during their Cleanup (equivalent, for this card set, to "since your last turn began").
- `--seed / --first` are non-RFC debug conveniences.
- Exile zone is tracked server-side but is not defined in the RFC; it was added for completeness since cards like Swords to Plowshares and Path to Exile remove permanents from the game permanently.

Work Distribution Matrix

Task / Feature	Brian Garcia	Jeremy Leano	Mark Canoso	Renzel Eleydo
TCP Server: connection handling, framing, dispatch	Lead		Support	
Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic	Lead	Support		
Turn & phase engine (all phases/steps, transitions)		Lead		Support
Priority & Stack logic, spell/ability resolution	Support	Lead		
Combat system (attackers, blockers, damage)		Lead	Support	
Card effects (all 58 cards, triggers, ETB, death)		Support	Lead	
Client implementation & state rendering			Lead	Support
PDU serialisation/deserialisation (all 25 PDU types)	Support			Lead
Error handling, PING/PONG heartbeat, disconnect logic			Support	Lead
Verbose mode (client + server PDU logging, toggle)	Support		Lead	
GUI web interface, spectator client (bonus features)		Support		Lead
Testing, README, documentation, bug fixes	Lead			Support

AI Usage Disclosure

The following AI tools were used during development:

- **Claude** (via Antigravity IDE) — used throughout the project as a coding assistant for:
 - Generating initial scaffolding for `server.py`, `client.py`, and `common.py` based on the RFC specification.
 - Implementing card effects and the combat system logic.
 - Designing and building the web GUI (`gui/index.html`, `gui_client.py`).
 - Writing the automated test suites (`test_game.py`, `test_combat.py`, `test_edge.py`).
 - Debugging issues such as the log freeze bug and the land detection bug.
 - Drafting this README document.

All AI-generated code was thoroughly reviewed, tested, and verified by all group members. Every member understands and can explain all parts of the codebase, including the TCP socket setup, message framing, game lifecycle state machine, priority/stack resolution, and combat system. AI was used as a productivity tool, not as a substitute for understanding the protocol and networking concepts.